EXP: 12(a) ECHO CLIENT-SERVER USING TCP.

24-09-24

AIM:-

To implement an Echo Client-Server application using TCP sockets and verify communication between the client and server.

PROCEDURE:-

1. Create a TCP Echo Server Using Server Socket and socket.
2. Assign a port number to the server.
3. Compile and execute the TCP Server program.
4. Create a TCP Echo Client using socket.
5. Connect the client to the server using the server address and post number.
6. Enter a message through the passage and send it back to the client.
7. The server recieves the passage and send it back to the client.
8. Display the echoed message at the client.
9. Enter bye to terminate the communication.

RESULT:-

Thus, the TCP Echo communication was successfully implemented and the client successfully recieved the echoed message from the server.

EXP: 12(a)
24-09-2c
Aim:-
To i
using
the

PROCED
1- Cre
so
2- A
3.
4.
5-

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter
PS C:\Users\Luqman\Downloads\CN_EXPT_12> javac TCPEchoServer.java TCPEchoClient.java
PS C:\Users\Luqman\Downloads\CN_EXPT_12> java TCPEchoServer
TCP Echo Server listening on port 5000
Client connected: /127.0.0.1:50613
Received: hello
Received: I am studying in REC
```

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter
PS C:\Users\Luqman\Downloads\CN_EXPT_12> java TCPEchoClient
Connected to TCP Echo Server at localhost:5000
Type messages to send (type 'bye' to quit):
hello
Server: Echo: hello
I am studying in REC
Server: Echo: I am studying in REC
```

TCPEcho server

```
import java.io;
import java.net;
public class chatServer{
    public static void ;
    s = socket.socket()
    s.bind("local host", 5000)
    s.listen(n)
    c, addr = s.accept()
    msg = c.recv(1024).
    c.send(msg)
    c.close()
    s.close().
}
```

TCPEcho Client.

```
import socket
s = socket.socket
s.connect(), "localhost", 5001,
s.send(b" Hello Serve")
print(s.recv(1024).decode()),
s.close()
```

EXP 12(b) ECHO-CLIENT-SERVER USING UDP.

24-09-26

AIM:-

To implement an Echo Client-Server using UDP, an application using UDPSockets and verify communicati on between Client and Server.

PROCEDURE:-

1. Create a UDP Echo Server using datagram socket and datagram packet.
2. Assign a port number to the UDP server.
3. Compile and execute the UDP server program.
4. Create a UDP Echo Client using datagram socket and datagram packet.
5. Specify the server address and port number.
6. Enter a message through the client.
7. The client sends the message as a UDP datagram to the server.
8. The server recieves the datagram and sends the echoed message back.
9. Display the recieved echoed message at the client.
10. Enter bye to terminate the communication.

RESULT:-

Thus, the UDP Echo Client-Server application was succesfully implemented, and the client succesfully recieved the echoed message from the server.

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

PS C:\Users\Luqman\Downloads\CN_EXPT_12> java UDPEchoClient
UDP Echo Client ready to send to localhost:6000
Type messages to send (type 'bye' to quit):
Hello
Server: Echo: Hello
I am studying Computer Networks
Server: Echo: I am studying Computer Networks
This is for Testing
Server: Echo: This is for Testing
```

```
Problems  Output  Debug Console  Terminal  Ports  Jupyter

PS C:\Users\Luqman\Downloads\CN_EXPT_12> javac UDPEchoServer.java UDPEchoClient.java
PS C:\Users\Luqman\Downloads\CN_EXPT_12> java UDPEchoServer
UDP Echo Server listening on port 6000
Received from /127.0.0.1:63830 -> Hello
Received from /127.0.0.1:63830 -> I am studying Computer Networks
Received from /127.0.0.1:63830 -> This is for Testing
```

CODE:-

UDPEcho Server:

```
import socket.
s = socket.socket(socket.AF_INET)
socket.sock_DRAM)
s.bind(("local host", 5000))
msg, addr = s.revfrom(1024)
s.send to (msg, addr)
s.close()
```

UDPEcho Client:

```
import socket.
s = socket.Socket(Socket.AF
socket.sock_DGRAM).
S.send to (b"Hello Server"
msg, addr = S.revfrom(1024
print(msg.decode())
s.close().
```

EXP 12(C) CHAT CLIENT-SERVER USING TCP/IP. SOCKETS.

24-09-26

AIM:-

To implement a chat client-server application using socket and enable communication between multiple clients through a server.

PROCEDURE:-

1. Create a chat server program using Socket programming.
2. Start and run the chat server.
3. Create the chat client program and connect it to the server.
4. Run two or more client program in separate terminals
5. Enter a name for each client.
6. Send message from one client to the server.
7. The server broadcasts the message to the other connected clients.
8. Verify that the clients are exchanging message through the server.
9. Continue the communication between the clients.
10. Enter/quit to terminate a client connection.

RESULT:-

Thus, the chat client-server application was successfully implemented and message was successfully exchanged blw multiple clients through the server.

```
PS C:\Users\Luqman\Downloads\CN_EXPT_12> java ChatServer
Chat server started on port 5000
Mr. X connected from /127.0.0.1
Mr. Y connected from /127.0.0.1
Mr. X: Hello, How are you?
Mr. Y: I am Fine
```

Problems Output Debug Console Terminal Ports Jupyter

```
PS C:\Users\Luqman\Downloads\CN_EXPT_12> java ChatClient
Enter your name:
Mr. X
Welcome, Mr. X! Type /quit to exit.
Mr. Y has joined the chat.
Hello, How are you?
Mr. Y: I am Fine
```

```
PS C:\Users\Luqman\Downloads\CN_EXPT_12> java ChatClient
Enter your name:
Mr. X has joined the chat.
Mr. Y
Welcome, Mr. Y! Type /quit to exit.
Mr. X: Hello, How are you?
I am Fine
```

EXP 12C

CODE:-

```
import java.io;
import java.net.*;
public class chatClient{
    public static void main(String args[])
        Scanner sc = new Scanner(System.in);
string host = "localhost";
int port = 5000;
Socket socket = new Socket(host,port);
BufferedReader in = new BufferedReader(
        socket.getInputStream()));

PrintWriter out = new PrintWrite(
        new OutputStreamWrite(socket.getOutput
                                Stream())).
try{
  String message;
  while(message = sc.nextline() != null){
      system.out.println(message);
  if(message.equalsIgnorecase("/quit"))
    break;
  }
socket.close()
System.out.println("Chat Client closed");
}
```